REPORT

Zajęcia: Analog and digital electronic circuits
Teacher: prof. dr hab. Vasyl Martsenyuk

Lab 7
Date 10.04.26
Topic: "Multidimensional Signal Processing:
Analysis of Image and Audio Signals"
Variant 13

Kostiantyn Hrekov
Informatyka II stopień,
stacjonarne,
1 semestr,
Gr 1.a

1. Problem statement:

Variant 13.

Each student is assigned a unique variant of the task. For each variant, students must:

- Synthetically generate the specified grayscale image and audio signal.
- Perform multidimensional signal analysis including Fourier transform, filtering, and visualization.
- Compare time and frequency domain representations.

Important: The image and audio must be generated in Python using NumPy, OpenCV, or other standard libraries.

Input data:

Image with only one vertical line in the center.

Audio: High-frequency sine burst (3000 Hz) with silence.

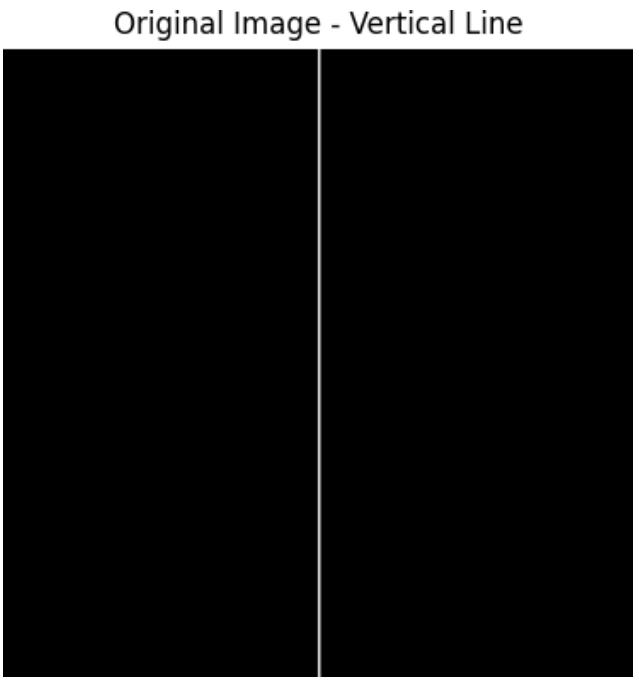
2. Commands used (or GUI):

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
from scipy.io.wavfile import write
import librosa
import librosa.display


# ==========================
# 1. GENERATE IMAGE
# ==========================
def generate_vertical_line(size=256):
    img = np.zeros((size, size), dtype=np.uint8)
    img[:, size // 2] = 255
    cv2.imwrite('vertical_line.png', img)
    return img


image = generate_vertical_line()

plt.imshow(image, cmap='gray')
plt.title("Original Image - Vertical Line")
plt.axis('off')
plt.show()

# ==========================
# 2. 2D FOURIER TRANSFORM
# ==========================
f = np.fft.fft2(image)
fshift = np.fft.fftshift(f)

magnitude_spectrum = 20 * np.log(np.abs(fshift) + 1)
```

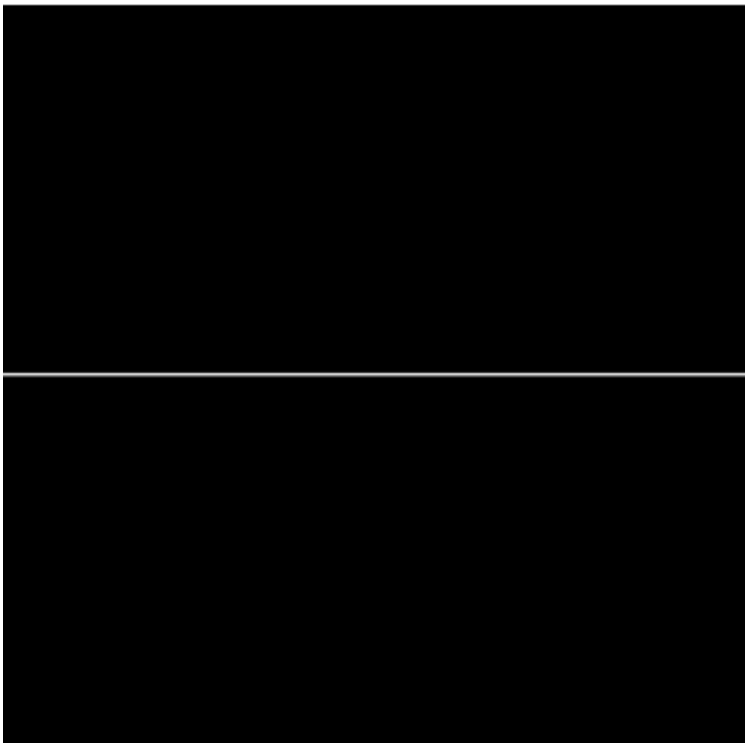
```
plt.imshow(magnitude_spectrum, cmap='gray')
plt.title("Magnitude Spectrum")
plt.axis('off')
plt.show()

# ==========================
# 3. FILTERING (LP + HP)
# ==========================
rows, cols = image.shape
crow, ccol = rows // 2, cols // 2

# Low-pass mask
mask_lp = np.zeros((rows, cols), np.uint8)
r = 30
cv2.circle(mask_lp, (ccol, crow), r, 1, -1)

# High-pass mask
mask_hp = 1 - mask_lp

# Apply filters
fshift_lp = fshift * mask_lp
fshift_hp = fshift * mask_hp

# Inverse FFT - LP
img_lp = np.fft.ifft2(np.fft.ifftshift(fshift_lp))
img_lp = np.abs(img_lp)

# Inverse FFT - HP
img_hp = np.fft.ifft2(np.fft.ifftshift(fshift_hp))
img_hp = np.abs(img_hp)

# Show results
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(img_lp, cmap='gray')
plt.title("Low-pass Image")
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(img_hp, cmap='gray')
plt.title("High-pass Image")
plt.axis('off')

plt.show()


# ==========================
# 4. GENERATE AUDIO (BURST)
# ==========================
def generate_burst(fs=44100, duration=2, freq=3000):
    t = np.linspace(0, duration, int(fs * duration))
    y = np.zeros_like(t)

    # burst only at beginning
    y[:fs // 5] = 0.5 * np.sin(2 * np.pi * freq * t[:fs // 5])

    write('burst.wav', fs, (y * 32767).astype(np.int16))
    return y, fs


y, sr = generate_burst()

# ==========================
# 5. TIME DOMAIN
# ==========================
plt.figure()
```

```
librosa.display.waveshow(y, sr=sr)
plt.title("Audio Signal (Time Domain)")
plt.show()

# ==========================
# 6. FREQUENCY DOMAIN
# ==========================
Y = np.fft.fft(y)
freqs = np.fft.fftfreq(len(Y), 1 / sr)

plt.figure()
plt.plot(freqs[:len(freqs) // 2], np.abs(Y[:len(Y) // 2]))
plt.title("Frequency Spectrum")
plt.xlabel("Frequency [Hz]")
plt.ylabel("Magnitude")
plt.show()

# ==========================
# 7. SPECTROGRAM (STFT)
# ==========================
D = librosa.stft(y)
S_db = librosa.amplitude_to_db(np.abs(D), ref=np.max)

plt.figure(figsize=(10, 4))
librosa.display.specshow(S_db, sr=sr, x_axis='time', y_axis='hz',
cmap='magma')
plt.colorbar(format='%+2.0f dB')
plt.title("Spectrogram (STFT)")
plt.tight_layout()
plt.show()
```

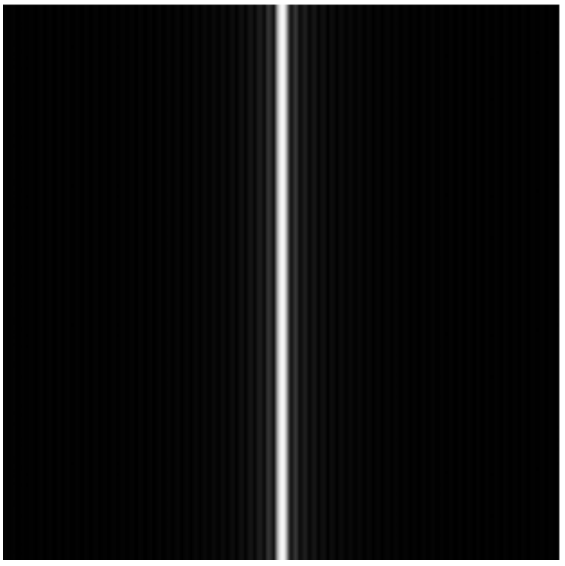
Output:



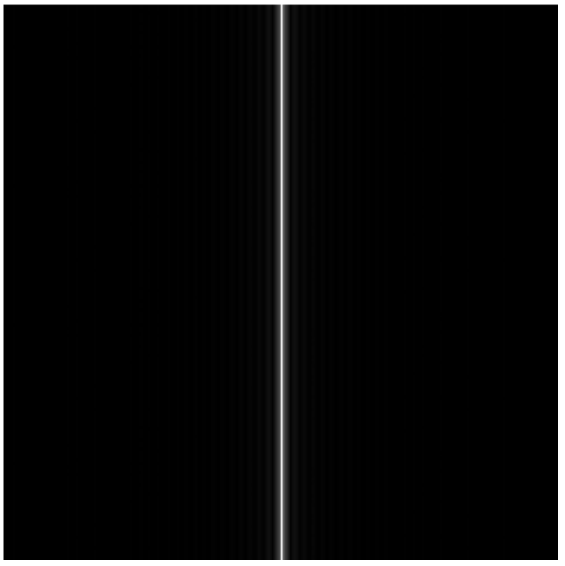
Magnitude Spectrum

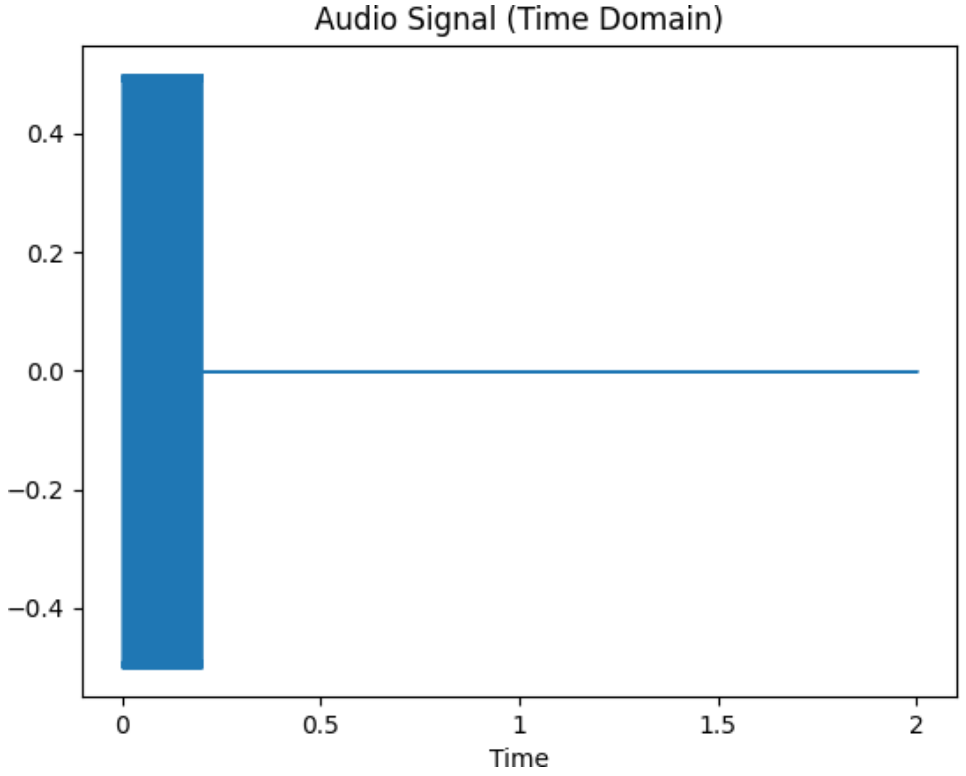


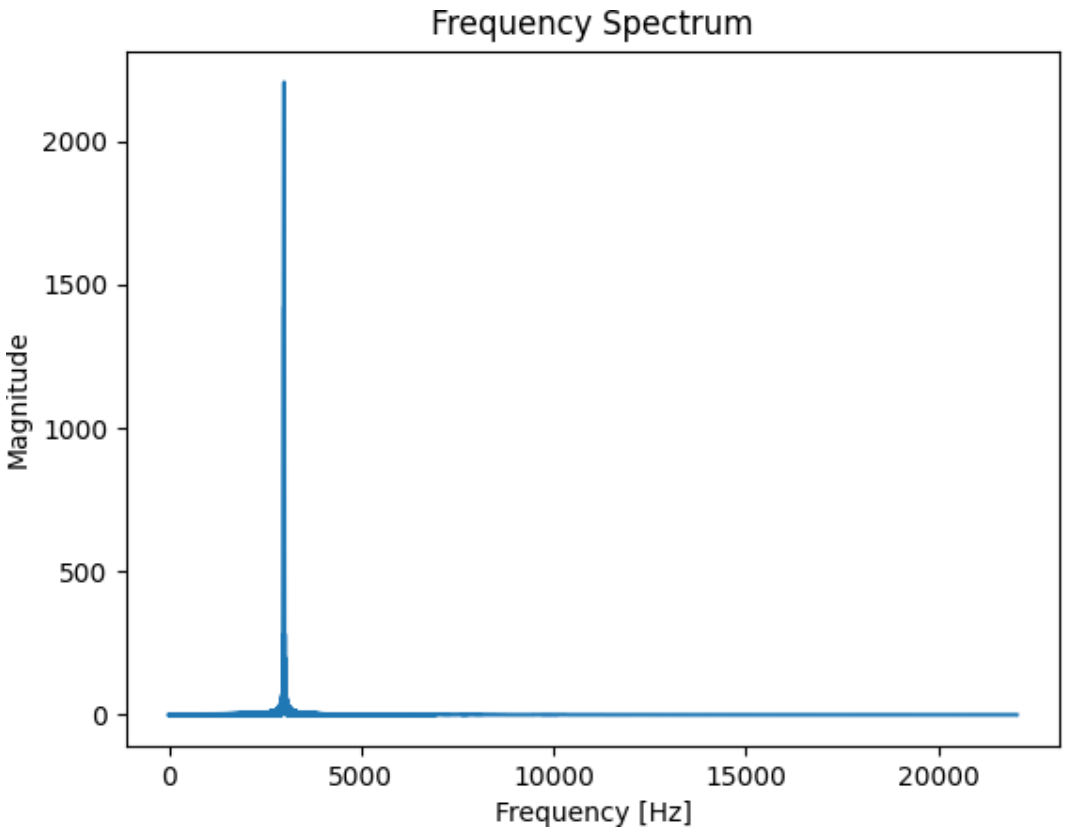
Low-pass Image

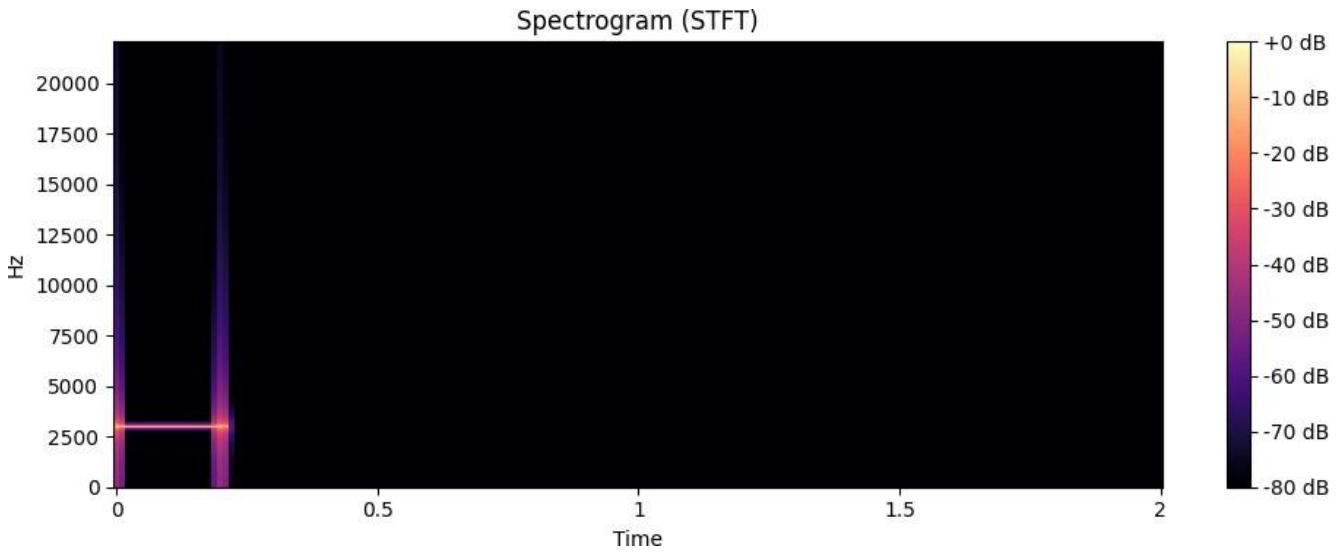


High-pass Image




Audio Signal (Time Domain)
0.4
0.2
0.0
−0.2
−0.4
0
0.5
1
1.5
2
Time


Frequency Spectrum
2000
1500
1000
500
0
Magnitude
0
5000
10000
15000
20000
Frequency [Hz]



Conclusion:

Due to this laboratory we learned that simple synthetic signals, such as a vertical line in an image and a short high-frequency burst in audio, show characteristic and easily interpretable patterns in the frequency domain. Additionally, the use of the Fourier Transform and STFT shows that both spatial and temporal features can be effectively analyzed, highlighting how signal structure and time localization influence their spectral representation.

Linkto remote repozytorium (e.g. GitHub):

https://github.com/KostiaGrekov/DSPja/tree/main/DSPja%207